

# RAP 後端開發練習 9：期末綜合實作 ( Capstone )

## Lecture

### 這一課要做什麼

這是這門課的期末總複習——不學新的 RAP 機制，而是把 rap02 ~ rap08 學過的技巧整合進同一個 **BO**，補完 rap08 那個「訂單」( Header-Item ) 情境，讓它具備完整的業務邏輯：

- **Determination ( 自動填值 )**：rap08 的 `create` 方法本來就會自動填 `created_at` / `created_by`，這已經是 Determination-equivalent 的技巧 ( 見 rap05 ) ——這一課直接沿用，不用重寫，另外多讓它自動填一個新欄位 `status` 的預設值。
- **Validation ( 資料驗證 )**：比照 rap06 的手寫等效寫法，新增規則「Item 的 `quantity` 必須大於 0」，不合法就整筆拒絕。
- **Action ( 商業操作 )**：比照 rap07，新增 `confirmOrder`——把訂單狀態從「New」改成「Confirmed」，而且加一條商業規則：沒有任何 Item 的訂單不能確認 ( 呼應 Composition 的語意——空訂單不是一張有意義的訂單 )。

這一課延伸的是 **rap08** 已經建好的物件，不是從零蓋一個新的資料模型——`ZRAP08_ORDER` / `ZRAP08_ORDER_I` ( 表格 )、`ZI_RAP08_ORDER` / `ZI_RAP08_ORDER_I` ( CDS View )、`ZI_RAP08_ORDER` ( BDEF )、`ZBP_I_RAP08_ORDER` ( 實作類別 ) 都在原本的基礎上加欄位、加方法，不用重新設計。跟 rap08 一樣，這些延伸由你在 **Eclipse** 動手做，**Claude** 負責事後驗證與除錯——不過這次 Claude 已經先用暫時性驗證物件 ( 複製 rap08 當初留下的 `ZRAP08V_H` / `ZRAP08V_I` / `ZI_RAP08VU_H` / `ZI_RAP08VU_I` / `ZBP_I_RAP08VU_H` ) 把新語法完整測過一輪，過程中踩到兩個坑 ( 下面會講 )，確保交給你的步驟是「這系統上真的能動」的版本。

### Part A：語法元素——這一課只新增三個小地方，其他全部是複習

1. 新欄位 `status`，用課程自訂的 **Domain + DE** ( 跟 rap02 的 `status`、rap08 的 `order_id` 一樣，找不到語意相符的標準 **Data Element**，比照建一組專屬的 )：

Claude 已經建立並啟用：

| 物件           | 名稱                            | 內容                                                                                                                                    |
|--------------|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Domain       | <code>ZRAP09_ORDSTATUS</code> | CHAR(1)，離散固定值： <code>N</code> = New、 <code>C</code> = Confirmed ( 第 15 節記載的離散固定值語法， <code>doma:fixValue</code> 不帶 <code>high</code> ) |
| Data Element | <code>ZRAP09_ORDSTATUS</code> | 標籤 <code>Status</code> / <code>Order Status</code>                                                                                    |

2. **Action** 帶商業規則，**RESULT** 要用 `%key + %param` ( 巢狀 )，不能扁平賦值——這是 rap07 已經教過的語法，這裡只是提醒一次：`APPEND VALUE #( %key = ls_key-%key %param = ls_data ) TO result.`，直接把整包資料結構丟給 `%param`，不要一個個欄位手動列出來 ( Claude 這次示範時第一次就寫成扁平欄位，啟用直接報 `No component exists with the name "DESCRIPTION"`，改成 `%param = ls_data` 才過 )。

3. ⚠️ ⚠️ 這一課踩到的關鍵坑：**BDEF** 裡直接宣告 **validation**，在 **Unmanaged** 非 **Draft** 完全不支援——這其實不是新發現，是 rap06 已經查證過官方文件確認的限制 ( "Not available for unmanaged, non-draft RAP BOs" )，這一課只是真的手滑寫了一次 `validation validateQuantity on save { field quantity; }`

進 **BDEF**，啟用報 `If specified, "validation" is only supported in the implementation type "managed"`。直接印證了 rap06 的結論。這一課的 **Validation** 一律用 rap06 教過的手寫等效寫法：在 `create_item` 方法內部用 `IF quantity <= 0` 判斷，不合法就 **APPEND** 到 `failed / reported`（用 `-item` 這個別名後綴，因為 `create_item` 是 **FOR MODIFY**，遵循 rap06 已經確認的規則：**FOR MODIFY** Handler 的 `failed / reported` 要加 `-<alias>`），不寫進 BDEF 的操作宣告清單。

完整程式碼：BDEF 修改

在 **ZI\_RAP08\_ORDER** ( BDEF ) 的 Header 區塊加 `action`，`field(readonly)` 加 `status`：

```
implementation unmanaged in class zbp_i_rap08_order unique;

define behavior for ZI_RAP08_ORDER alias Header
lock master
{
    create;
    update;
    delete;

    association _Item { create; }

    action confirmOrder result [1] $self;

    field ( readonly ) created_at, created_by, status;
}

define behavior for ZI_RAP08_ORDER_I alias Item
lock dependent ( order_id = order_id )
{
    update;
    delete;

    field ( readonly ) order_id;

    association _Header;
}
```

**Item** 區塊完全不用改——`quantity` 的驗證邏輯寫在 Header 的 `create_item` 方法裡（Item 是透過 Header 的 Create-by-Association 建立的，驗證邏輯自然也放在同一個方法裡檢查），BDEF 的 Item 操作宣告不需要新增任何東西。

完整程式碼：CDS View 修改

**ZI\_RAP08\_ORDER** 欄位清單加 `status`：

```
@AbapCatalog.sqlViewName: 'ZIRAP08ORDER'
@AbapCatalog.preserveKey: true
```

```

@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'RAP08 Order Header View'
@ObjectModel.compositionRoot: true
@Metadata.allowExtensions: true
define root view ZI_RAP08_ORDER
  as select from zrap08_order

  composition [0..*] of ZI_RAP08_ORDER_I as _Item
{
  key order_id,
  description,
  status,
  created_at,
  created_by,

  _Item
}

```

完整程式碼：Table 修改

ZRAP08\_ORDER 加一個欄位：

```

@EndUserText.label : 'RAP08 Order Header Table'
@AbapCatalog.enhancementCategory : #EXTENSIBLE_ANY
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #LIMITED
define table zrap08_order {
  key client      : mandt not null;
  key order_id    : zrap08_orderid not null;
  description     : text100;
  status          : zrap09_ordstatus;
  created_at      : timestamp1;
  created_by      : syuname;

}

```

完整程式碼：實作類別修改

ZBP\_I\_RAP08\_ORDER 的 Local Types Include ( `lhc_header` 這個類別 ) 要做四處修改：

1. **create** 方法：新建訂單時預設 `status = 'N'` ( 這就是這一課的 Determination-equivalent——沿用 rap08 create 方法本來就有的自動填值機制，多填一個欄位而已 )。
2. **create\_item** 方法：新增 Validation-equivalent 邏輯，`quantity <= 0` 就拒絕這筆 Item。
3. 新增 **confirmOrder** 方法 ( Action Handler )：檢查訂單底下有沒有 Item，沒有就拒絕；有的話把 `status` 改成 'C'，回傳更新後的資料。

4. **read\_header** 方法：**SELECT** 跟 **APPEND VALUE** 都要加上 **status** 欄位，不然畫面查不到這個欄位的值。

```
CLASS lhc_header DEFINITION INHERITING FROM cl_abap_behavior_handler.
PRIVATE SECTION.
METHODS lock FOR LOCK
IMPORTING it_lock FOR LOCK Header.

METHODS create FOR MODIFY
IMPORTING it_create FOR CREATE Header.

METHODS create_item FOR MODIFY
IMPORTING it_create FOR CREATE Header\_Item.

METHODS update FOR MODIFY
IMPORTING it_update FOR UPDATE Header.

METHODS delete FOR MODIFY
IMPORTING it_delete FOR DELETE Header.

METHODS confirmOrder FOR MODIFY
IMPORTING keys FOR ACTION Header~confirmOrder RESULT result.

METHODS read_header FOR READ
IMPORTING it_read FOR READ Header RESULT et_result.
ENDCLASS.

CLASS lhc_header IMPLEMENTATION.

METHOD lock.
ENDMETHOD.

METHOD create.
LOOP AT it_create INTO DATA(ls_create).
INSERT zrap08_order FROM @( VALUE #(
client      = sy-mandt
order_id    = ls_create-order_id
description = ls_create-description
status      = 'N'
created_at  = cl_abap_tstmp=>utclong2tstmp( utclong_current( ) )
created_by  = sy-uname ) ).
ENDLOOP.
ENDMETHOD.

METHOD create_item.
LOOP AT it_create INTO DATA(ls_create).
```

```

LOOP AT ls_create-%target INTO DATA(ls_target).
  IF ls_target-quantity <= 0.
    APPEND VALUE #( %cid = ls_target-%cid ) TO failed-item.
    APPEND VALUE #( %cid = ls_target-%cid
                    %msg = new_message_with_text(
                        severity = if_abap_behv_message=>severity-error
                        text      = 'Quantity must be greater than zero' ) )
    TO reported-item.
  CONTINUE.
ENDIF.

```

```

INSERT zrap08_order_i FROM @( VALUE #(
  client      = sy-mandt
  order_id    = ls_create-%key-order_id
  item_id     = ls_target-item_id
  material_desc = ls_target-material_desc
  quantity    = ls_target-quantity ) ).

```

ENDLOOP.

ENDLOOP.

ENDMETHOD.

METHOD update.

```

LOOP AT it_update INTO DATA(ls_update).

```

```

  UPDATE zrap08_order SET description = @ls_update-description
  WHERE order_id = @ls_update-%key-order_id.

```

ENDLOOP.

ENDMETHOD.

METHOD delete.

```

LOOP AT it_delete INTO DATA(ls_delete).

```

```

  DELETE FROM zrap08_order_i WHERE order_id = @ls_delete-%key-order_id.

```

```

  DELETE FROM zrap08_order WHERE order_id = @ls_delete-%key-order_id.

```

ENDLOOP.

ENDMETHOD.

METHOD confirmOrder.

```

LOOP AT keys INTO DATA(ls_key).

```

```

  SELECT COUNT(*) FROM zrap08_order_i WHERE order_id = @ls_key-%key-order_id INTO
  @DATA(lv_count).

```

```

  IF lv_count = 0.

```

```

    APPEND VALUE #( %key = ls_key-%key ) TO failed-header.

```

```

    APPEND VALUE #( %key = ls_key-%key

```

```

                    %msg = new_message_with_text(

```

```

                        severity = if_abap_behv_message=>severity-error

```

```

                        text      = 'Cannot confirm an order with no items' ) )

```

```

    TO reported-header.

```

```

        CONTINUE.
    ENDIF.

    UPDATE zrap08_order SET status = 'C' WHERE order_id = @ls_key-%key-order_id.
ENDLOOP.

LOOP AT keys INTO ls_key.
    SELECT SINGLE order_id, description, status, created_at, created_by
        FROM zrap08_order WHERE order_id = @ls_key-%key-order_id INTO @DATA(ls_data).
    IF sy-subrc = 0.
        APPEND VALUE #(
            %key      = ls_key-%key
            %param = ls_data ) TO result.
    ENDIF.
ENDLOOP.
ENDMETHOD.

METHOD read_header.
    LOOP AT it_read INTO DATA(ls_key).
        SELECT SINGLE order_id, description, status, created_at, created_by
            FROM zrap08_order WHERE order_id = @ls_key-order_id INTO @DATA(ls_data).
        IF sy-subrc = 0.
            APPEND VALUE #(
                %key          = ls_key-%key
                order_id      = ls_data-order_id
                description    = ls_data-description
                status        = ls_data-status
                created_at    = ls_data-created_at
                created_by    = ls_data-created_by ) TO et_result.
        ENDIF.
    ENDLOOP.
ENDMETHOD.

ENDCLASS.

```

**lhc\_item** 這個類別完全不用改——**update / delete / read\_item** 維持 rap08 解答段落已經補完的版本。

✅ 驗證結果 ( Claude 用暫時性驗證物件，**programrun** 無頭執行，三個情境全部成功 )

```

=== Scenario 1: confirm order with 0 items -> should FAIL ===
confirm V001 failed: X
    message: Cannot confirm an order with no items
=== Scenario 2: confirm order with 1 valid item -> should SUCCEED ===
create V002+item failed:
confirm V002 failed:
    status after confirm: C

```

```
DB status: C
=== Scenario 3: create item with quantity <= 0 -> should FAIL at create ===
create V003 header failed:
create V003 item failed: X
    item message: Quantity must be greater than zero
    item rows for V003 in DB:          0
```

三個情境分別驗證了：Action 帶商業規則正確擋下空訂單、正常流程（建立訂單 + Item→確認）完整成功且狀態正確更新、Validation-equivalent 正確擋下不合法的 Item 且完全沒有寫進資料庫。

Eclipse ADT：延伸 rap08 的物件——Step by Step（輪到你了）

這一課不是建全新物件，是回頭修改你在 **rap08** 已經建好的四個物件，跟着下面步驟操作：

### 1. 修改 Table **ZRAP08\_ORDER**

雙擊打開既有的 **ZRAP08\_ORDER**，在 **description** 跟 **created\_at** 之間插入一行：

```
status          : zrap09_ordstatus;
```

存檔（Ctrl+S）+ 啟用。如果啟用報找不到 **ZRAP09\_ORDSTATUS** 這個型別，代表 Claude 建立的 Domain / DE 還沒同步到你的 Eclipse 快取，按一下 Eclipse 的重新整理（F5）再試一次。

### 2. 修改 CDS View **ZI\_RAP08\_ORDER**

在 **description** 跟 **created\_at** 之間插入 **status**，這一行（整份程式碼比對上方「完整程式碼：CDS View 修改」）。存檔 + 啟用。

### 3. 修改 Behavior Definition **ZI\_RAP08\_ORDER**

比對上方「完整程式碼：BDEF 修改」，Header 區塊加 **action confirmOrder result [1] \$self**；這一行，**field ( readonly )** 那行補上 **status**。存檔 + 啟用。

⚠ 如果你手滑照著官方教材的印象寫成 **validation** 語法，會啟用失敗——這一課的 Validation 不寫進 BDEF，見上面 Part A 第 3 點的說明。

### 4. 修改實作類別 **ZBP\_I\_RAP08\_ORDER**

Ctrl+1 在 BDEF 新增的 **confirmOrder** 那一行上，選 **Add missing behavior implementations**（或類似字樣的快速修正選項），Eclipse 會自動幫你在 **lhc\_header** 生成一個空的 **confirmOrder** 方法骨架——如果你的 Eclipse 版本沒有跳出這個選項，直接手動在 **lhc\_header** 的 **DEFINITION / IMPLEMENTATION** 區塊照抄上方「完整程式碼：實作類別修改」的 **confirmOrder** 方法宣告 + 實作即可。

接著比對上方完整程式碼，依序修改：

1. **create** 方法：新增 **status = 'N'** 這一行
2. **create\_item** 方法：在 **LOOP AT ls\_create-%target** 裡面、**INSERT** 之前，加上 **IF ls\_target-quantity <= 0**。那一段驗證邏輯

3. `read_header` 方法：`SELECT` 欄位清單跟 `APPEND VALUE` 都加上 `status`

存檔 + 啟用整個類別。

## 5. 通知 Claude 驗證

完成以上四個步驟後跟 Claude 說一聲，Claude 會：

1. 讀取比對你修改的四個物件內容
2. 確認 `sap_inactive_objects` 沒有殘留
3. 建立一支驗證程式 ( `ZR_RAP09_DEMO` )，用 `programrun` 跑上面三個情境，確認行為完全一致

加碼：讓 `status` 顯示、`confirmOrder` 出現按鈕——更新 Metadata Extension

`rap08` 建的 Metadata Extension ( `ZI_RAP08_ORDER` ) 原本沒有處理 `status` 欄位跟 `confirmOrder` Action，Preview 打得開但看不到 `status`，也沒有按鈕能觸發 Action。Claude 已經用暫時性驗證物件把下面這套語法測過一輪，一次就啟用成功，等你完成上面的 Table / CDS View / BDEF / 類別延伸後，Claude 會直接更新這個 Metadata Extension。

新語法：`@UI.identification` 陣列可以放多筆，其中一筆用 `type: #FOR_ACTION` 就會變成按鈕 ( 查證 SAP 官方文件 \*Exposing the Action Extension for UI Consumption\* 確認 )：

- `{ type: #FOR_ACTION, dataAction: '<Action 名稱>', label: '<按鈕文字>' }`：放進任一欄位的 `identification` 陣列裡 ( 習慣上放在 Object Page 上排第一順位的欄位，例如 `order_id` )，Fiori Elements 就會在 Object Page ( 有些情況也會在 List Report 工具列 ) 生出一顆對應的按鈕。`dataAction` 填 Action 在 BDEF 裡宣告的名稱 ( `confirmOrder` )，不用加任何路徑前綴。
- 同一個 `identification` 陣列可以同時有「一般欄位識別」跟「Action 按鈕」兩種條目——`order_id` 原本就有 `{ position: 10 }` 這筆 ( 讓它出現在 Object Page 的欄位清單裡 )，現在多加一筆 `{ type: #FOR_ACTION, ... }`，兩者並存，互不影響。
- `status` 欄位維持跟其他欄位一樣的 `@UI.lineItem` / `@UI.identification` 標記即可，不需要額外做什麼特殊處理就會正常顯示技術值 ( `N / C` )；想要用顏色 / 圖示區分 New / Confirmed ( `criticality` )，需要另外設計一個「回傳 0~3 數值」的計算欄位讓 `criticality` 參照，這一課先不做，算是進階選做。

更新後的完整 Metadata Extension：

```
@Metadata.layer: #CUSTOMER
@UI: {
  headerInfo: {
    typeName: 'Order',
    typeNamePlural: 'Orders',
    title: { type: #STANDARD, value: 'order_id' }
  }
}

annotate view ZI_RAP08_ORDER with
{
  @UI.facet: [
```

```

    { id: 'GeneralInformation', purpose: #STANDARD, type: #IDENTIFICATION_REFERENCE,
label: 'General Information', position: 10 },
    { id: 'Items', purpose: #STANDARD, type: #LINEITEM_REFERENCE, label: 'Order
Items', targetElement: '_Item', position: 20 }
  ]

  @UI: {
    lineItem: [{ position: 10 }],
    selectionField: [{ position: 10 }],
    identification: [
      { position: 10 },
      { type: #FOR_ACTION, dataAction: 'confirmOrder', label: 'Confirm Order' }
    ]
  }
  order_id;

  @UI.lineItem: [{ position: 20 }]
  @UI.identification: [{ position: 20 }]
  description;

  @UI.lineItem: [{ position: 30 }]
  @UI.identification: [{ position: 30 }]
  status;

  @UI.lineItem: [{ position: 40 }]
  @UI.identification: [{ position: 40 }]
  created_at;

  @UI.lineItem: [{ position: 50 }]
  @UI.identification: [{ position: 50 }]
  created_by;
}

```

跟原本 rap08 的版本比，差異只有兩處：`order_id` 的 `@UI.identification` 改成陣列形式、多一筆 `FOR_ACTION`；新增 `status` 這個欄位的標記（跟其他欄位同樣寫法）。

## Fiori Elements Preview / Postman 重新測試

rap08 已經建好 `ZRAP08_SD` / `ZRAP08_SB` 並 Publish 過，`status` 欄位跟 `confirmOrder` Action 加進去之後不需要重新建 **Service Binding**，Publish 過的服務會自動反映新的欄位與操作（如果 Preview 沒有立刻更新，重新整理瀏覽器分頁即可）。Metadata Extension 更新後：

**Preview (Fiori Elements)**：`status` 欄位會正常顯示在 List Report / Object Page；點進任一筆訂單的 Object Page，應該會看到一顆 **Confirm Order** 按鈕，點下去就是真正呼叫 `confirmOrder` Action——沒有 Item 的訂單點了會跳出錯誤訊息（`Cannot confirm an order with no items`），有 Item 的訂單點了會成功、`status` 變成 `Confirmed`。

Postman：

- 讀取 ( `GET .../Orders` ) 跟寫入 ( `POST / PUT / DELETE` ) 語法完全比照 rap08 Part D 教過的方式，`status` 欄位會直接出現在回傳的 JSON 裡。
- ⚠️ `confirmOrder` 這個 Action，不是 rap08 教的 `POST .../Orders(...)` 那種 Entity CRUD 語法——OData V2 沒有 V4 那種原生 Bound Action 概念，RAP 框架的轉譯層會把 Action 包成傳統的 **Function Import** ( 語法通常是 `POST /ServiceName/FunctionImportName?參數='值'` )，確切的 **Function Import** 名稱、要帶什麼參數，等你把這一課的修改建好、**Service** 重新整理過中繼資料後，一定要先 `GET .../$metadata` 查證，不要用猜的——這是 rap08 Part D 已經教過的原則，這裡再次適用。查到正確語法後，可以測：
  - 建一筆沒有 Item 的訂單，呼叫 `confirmOrder`，確認回應是失敗 ( 帶 `Cannot confirm an order with no items` 訊息 )
  - 建一筆有 Item 的訂單再呼叫，確認成功、`status` 變成 `C`

## Part D：加碼——這系統 ( S/4HANA 1909，Unmanaged 非 Draft ) 標準「子表格 Create 按鈕」不可靠，改用 Custom Action 解決

問題是怎麼發現的

實際在 Eclipse Preview 測試時，一開始 Object Page 完全沒有出現「**Order Items**」這個分頁區塊——查 `$metadata` 才發現：Service Definition 只 `expose` 了 Root ( `ZI_RAP08_ORDER` )，`_Item` 完全沒有變成 OData 的 Navigation Property ( 第 40 節提過的「已知結論」原來是沒實測過的推論，這裡被推翻並已回頭修正 rap08 Part D )。補上 `expose ZI_RAP08_ORDER_I as Items;` 之後，`$metadata` 正確出現了 `NavigationProperty / Association / AssociationSet`，Object Page 也真的出現了「Order Items」分頁跟表格。

但接下來測「點 Edit → 表格裡有沒有 Create 按鈕」，結果按了 **Edit** 之後，連原本顯示模式下有的 **Delete** 按鈕都消失了，**Create** 也沒有出現，連 **Key** 欄位 **Order ID** 都被畫面標成「必填、可編輯」( 正常來說已存在資料的 Key 欄位不該讓人改 )。查證官方文件 ( \*Generic Action Buttons in Tables on the Object Page\* ) 確認「Create 按鈕只在 Edit 模式才顯示」的規則沒錯，但這系統的 Edit 模式本身在這個 Unmanaged + 非 Draft 的組合下，表現跟官方文件描述的 ( 以及大部分 Draft-enabled 範例展示的 ) 不一致。

結論：這是「S/4HANA 1909 + Unmanaged 非 Draft」這個組合的已知限制，不是語法寫錯

繼續深究這個 Edit 模式異常行為，需要的是 UI5 / Fiori Elements 前端框架層級的除錯 ( 不是 RAP 後端語法問題 )，已經超出這門課「RAP 後端開發」的範圍 ( README 從一開始就寫明「真正的 Fiori Elements 畫面設計... 不在本課程範圍內」 )。這系統這個版本的組合 ( **S/4HANA 1909**、**Unmanaged**、**非 Draft** ) 沒有可靠的方法讓使用者直接在 **Object Page** 子表格裡用標準 **Create** 按鈕新增 Item——這是這門課到目前為止唯一一個「後端邏輯完全正確、OData 曝露也完全正確，但標準 Fiori Elements UI 元件不可靠」的案例，值得記錄下來。

解法：把「新增 Item」改成一個獨立的 Custom Action ( `addItem` )，繞開整個標準 Create 機制

Custom Action ( `type: #FOR_ACTION` ) 在這門課從 rap07 開始就一直穩定可靠 ( `confirmOrder` 已經證實可以正常顯示成按鈕 )——既然標準 **Create** 按鈕靠不住，乾脆不依賴它，改用一個帶參數的 **Action** 達到同樣效果：使用者點一顆「Add Item」按鈕，Fiori Elements 自動彈出參數輸入視窗 ( 填 `item_id / material_desc / quantity` )，確認送出後在後端直接執行新增邏輯——體驗上功能一樣，只是換一條更可靠的實作路徑。

新語法：`action ... parameter <CDS Abstract Entity> result [1] $self;` (查證官方 ABAP 語言文件 `ABENBDL_ACTION2_ABEXA` 確認)——這門課到目前為止的 Action ( `confirmOrder` ) 都沒有輸入參數，這是第一次需要讓使用者「填資料」再執行 Action：

- **parameter <名稱>**：宣告這個 Action 需要輸入參數，<名稱> 指向一個 **CDS Abstract Entity** ( **define abstract entity** ) ——這是一種專門用來「定義參數形狀」的輕量 CDS 物件，沒有底層資料庫表格、不能單獨查詢，純粹描述「這個 Action 要收哪些欄位」。
- **Handler Method 裡讀取參數**：**keys** 這個 Importing 參數，每一列除了原本的 **%key** ( 識別要對哪個 Header 執行 )，現在多了 **%param**——結構跟 Abstract Entity 定義的欄位一致，用 **ls\_key-%param-<欄位名>** 讀取使用者填的值。
- **EML 呼叫端對應**：**EXECUTE addItem FROM VALUE #( ( %key-order\_id = 'R904' %param-item\_id = '0010' %param-material\_desc = '...' %param-quantity = '3' ) )**——**%key-** 前綴指定要操作哪筆 Header，**%param-** 前綴帶入這次呼叫的輸入參數，兩者都在同一列裡。

### Abstract Entity ( 參數定義 ) ZI\_RAP08\_ADDITEM :

```
@EndUserText.label: 'RAP08 AddItem Action Parameters'
define abstract entity ZI_RAP08_ADDITEM
{
    item_id          : zrap08_itemid;
    material_desc    : text100;
    quantity         : zrap08_quantity;
}
```

三個欄位直接重用既有的 Data Element ( `zrap08_itemid` / `text100` / `zrap08_quantity` )，不用額外建新的。

**BDEF 的 Header 區塊新增一行：**

```
action addItem parameter ZI RAP08 ADDITEM result [1] $self;
```

### Handler Method :

```
METHODS addItem FOR MODIFY
  IMPORTING keys FOR ACTION Header~addItem RESULT result.
```

```
METHOD addItem.  
  LOOP AT keys INTO DATA(ls_key).  
    IF ls_key-%param-quantity <= 0.  
      APPEND VALUE #( %key = ls_key-%key ) TO failed-header.  
      APPEND VALUE #( %key = ls_key-%key  
                      %msg = new_message_with_text(  
                        severity = if_abap_behv_message=>severity-error  
                        text      = 'Quantity must be greater than zero' ) )  
        TO reported-header.  
    CONTINUE.
```

```

ENDIF.

INSERT zrap08_order_i FROM @( VALUE #(
    client      = sy-mandt
    order_id    = ls_key-%key-order_id
    item_id     = ls_key-%param-item_id
    material_desc = ls_key-%param-material_desc
    quantity    = ls_key-%param-quantity ) ).
ENDLOOP.

LOOP AT keys INTO ls_key.
    SELECT SINGLE order_id, description, status, created_at, created_by
        FROM zrap08_order WHERE order_id = @ls_key-%key-order_id INTO @DATA(ls_data).
    IF sy-subrc = 0.
        APPEND VALUE #(
            %key    = ls_key-%key
            %param  = ls_data ) TO result.
    ENDIF.
ENDLOOP.
ENDMETHOD.

```

邏輯上跟 `create_item` 幾乎一樣（同樣的 `quantity` 驗證、同樣的 `INSERT`），差別只在資料來源——`create_item` 是從 Create-by-Association 的 `%target` 拿資料，`addItem` 是從 Action 的 `%param` 拿資料。

**Metadata Extension：**`addItem` 按鈕放在跟 `confirmOrder` 同一個位置（`order_id` 的 `identification` 陣列）——一個欄位的 `identification` 陣列可以放任意多筆 `FOR_ACTION` 條目：

```

identification: [
    { position: 10 },
    { type: #FOR_ACTION, dataAction: 'confirmOrder', label: 'Confirm Order' },
    { type: #FOR_ACTION, dataAction: 'addItem', label: 'Add Item' }
]

```

✅ 驗證結果（`programrun` 無頭執行，真實物件，完全成功）

```

=== Scenario 4: addItem on an existing order, then confirm ===
create R904 failed:
addItem (bad quantity) failed: X
    message: Quantity must be greater than zero
    item rows for R904 after bad addItem:                0
addItem (valid) failed:
    item rows for R904 after good addItem:                1
    item material_desc:
Good Widget
quantity:        3.00

```

```
confirm R904 (after addItem) failed:
final DB status: C
```

流程：建立一筆沒有 Item 的訂單 → 先試不合法的 `quantity (-1)`，正確被拒絕、完全沒寫入資料庫 → 再試合法的 `quantity (3)`，成功建立 → 這時候 `confirmOrder` 終於能成功執行、`status` 變成 `Confirmed`——完整串起了這一課三個機制 (Validation-equivalent、新的 Action、既有的 Action) 的端對端流程。

這部分由 Claude 直接實作，不是使用者 Eclipse 步驟

⚠ 跟這一課前面的 **Determination / Validation / Action** ( `status` / `confirmOrder` ) 不一樣：Part D 是在你已經完成正課驗收後，臨場排查「Object Page 子表格 Create 按鈕不可靠」這個問題時發現的解法，屬於**除錯 / 補強**性質，不是這一課原本規劃要你動手練習的核心內容——所以這五個物件 (Abstract Entity `ZI_RAP08_ADDITEM`、BDEF 的 `addItem` Action、`ZBP_I_RAP08_ORDER` 的 `addItem` Handler、Service Definition 的 `expose Items`、Metadata Extension 的 `addItem` 按鈕) 都是 **Claude** 直接建立 / 修改並啟用的，沒有比照前面的分工原則交給你在 Eclipse 操作。

如果你想練習「帶參數 Action」這個新語法 (這一課第一次教)，可以回 Eclipse 打開 `ZI_RAP08_ORDER` 的 BDEF / `ZBP_I_RAP08_ORDER` / 新的 `ZI_RAP08_ADDITEM`，對照上面的完整程式碼核對內容，當作事後複習；不想練習也沒關係，Claude 已經用 `programrun` 完整驗證過，功能是正常的。

## Part E：加碼——`addItem` 按鈕的兩個 UI 行為，以及 Key 欄位在 Edit 模式被誤標可編輯的解法

驗收完 Part D 之後，實際在 Eclipse Preview 操作 `addItem` 按鈕時，又發現三個值得記錄的行為。

Q1：為什麼 `addItem` 按鈕不用先按 Edit 就能點？

`confirmOrder` / `addItem` 這種掛在 `@UI.identification` 的 `type: #FOR_ACTION` 按鈕，跟 Part D 一開始想用的「子表格標準 Create 按鈕」是兩套完全不同的機制：

- **標準 Create / Delete 按鈕**：綁定在實體的 `create` / `delete` 標準操作上，Fiori Elements 的慣例是「先進入 Edit 模式，才能新增/刪除資料」——這正是 Part D 一開始踩到的限制 (這系統的 Edit 模式在 Unmanaged 非 Draft 底下不可靠)。
- **Custom Action 按鈕 (FOR\_ACTION)**：Action 在 RAP 的定位是「對既有資料執行一個操作」，不屬於「進入編輯狀態、暫存變更、按 Save 才真正送出」這種 Draft/Edit 語意——**Action 按鈕點下去就是立即呼叫** (等同直接送出 EML `EXECUTE`)，不需要、也不會被 Edit 模式管控。這是 Custom Action 相較標準 Create 按鈕的一個額外好處：不只繞開了這系統的 Edit 模式限制，行為本身也更直覺 (不用多一步「先進 Edit」)。

Q2：為什麼新增的 Item 要手動 Refresh 才會出現在子表格？

`addItem` 呼叫成功後，畫面沒有自動把新資料加進「Order Items」表格，要手動重新整理頁面才看得到——這不是 bug，是 RAP / Fiori Elements 一個叫 **Side Effect (副作用)** 的機制在起作用：

- Fiori Elements 預設只有在「Action 回傳的實例，跟畫面當下綁定的實例不是同一個」時，才會自動觸發局部重新整理。
- `addItem` 的宣告是 `result [1] $self`——`$self` 代表「回傳的還是 Header 自己這個實例」，跟畫面上原本就綁定的 Header 實例是同一個，所以框架判斷「沒有變化需要刷新」，不會主動去重新查詢

`_Item` 子表格。

- 官方有一個宣告式的解法：在 BDEF 加 `side effects { action addItem affects entity _Item; }`，明確告訴框架「這個 Action 執行後，`_Item` 這個關聯的資料要重新讀取」。但這條路目前用不了——Side Effect 要真正生效，OData V2 這邊需要額外的 Gateway 層設定（Local Annotation File 或 `*_MPC_EXT` 類別），屬於 SEGW / IWBEF 的技術範圍，不是純 RAP / CDS 能決定的，已經超出這門課「RAP 後端」的範圍，不繼續深入。

**實務結論：**這是目前這個系統上的一個已知限制，手動重新整理（F5 或重新查詢列表）是唯一可靠的 workaround，不影響資料本身正確性（`programrun` 驗證已經確認 `addItem` 真的把資料寫進資料庫了，只是畫面沒有自動反映）。

Q3：Header 的 `ORDERID`、Item 的 `ITEM ID` 進 Edit 模式後可以被改，怎麼擋？

這是繼 Part D 之後，又一個「這系統的 BDL 剖析器停在較舊語言版本」的案例，過程也順便驗證出一個比原本設想更好的解法。

✗ 官方現行語法 `field(readonly:update)` 這系統不支援：官方文件（`ABENBDL_FIELD_CHAR`）明確列出 `field(readonly:update) field_e;` 這種帶冒號的「限定某個操作才生效」寫法（`readonly:create / readonly:update / mandatory:create / mandatory:update`），概念上正是「Create 時可填、Update 時不能改」——完全對應這裡想要的效果。但這系統的 BDL 剖析器對這個寫法一律報語法錯誤：

```
) | , " expected, not ":".  
( | ; " expected, not ")".
```

跟這門課一路遇到的 `obsolete` 語法問題（`strict / view entity / etag master`）是同一個模式：這系統的 BDL 剖析器版本比官方現行文件描述的舊。

✓ 但 `field(features:instance)` 這個動態欄位控制語法可以用——這是官方文件裡另一個獨立的機制（動態、依實例狀態決定欄位限制，不是靜態的操作限定），語法上剛好也是 `field(關鍵字:值)` 這個帶冒號的格式，一開始不確定會不會踩到同樣的限制，實測發現這個語法完全支援，只是搭配的 Handler Method 宣告要用 `FOR FEATURES`（不含 `INSTANCE`——這系統對可選的 `INSTANCE` 這個字報 `"INSTANCE" is not valid.`，拿掉就編譯成功）：

```
" BDEF : Header  
field ( features : instance ) order_id;  
  
" BDEF : Item  
field ( features : instance ) item_id;
```

```
" 實作類別 ( Header )  
METHODS get_instance_features FOR FEATURES  
    IMPORTING keys REQUEST requested_features FOR Header RESULT result.  
...  
METHOD get_instance_features.  
    result = VALUE #( FOR ls_key IN keys  
        ( %key          = ls_key-%key
```

```
%field-order_id = if_abap_behv=>fc-f-read_only ) ).  
ENDMETHOD.
```

Item 實體 ( `lhc_item` ) 也要各自補一組同樣的 `get_instance_features / FOR FEATURES / %field-item_id`，因為欄位層級的動態控制是每個實體各自宣告、各自實作，Header 補了不會自動套用到 Item。

為什麼這樣就能達到「Create 可填、Update 不能改」的效果：FOR FEATURES 這個 Handler 只有在框架要查詢「已存在的實例」該有什麼欄位限制時才會被呼叫——新建立中、還沒有 Key 的實例 ( Create 階段 ) 根本不會觸發這個查詢，所以欄位在 Create 畫面照樣是可填的；只有查詢/編輯已經存在的實例 ( 也就是 Update 情境 ) 時，框架才會呼叫這個方法、拿到 `read_only`，把欄位鎖住。這跟靜態的 `readonly:update` 想達成的效果在語意上是等價的，只是實現機制從「宣告式的靜態規則」換成「依實例狀態動態回答」。

已用 `checkruns / sap_inactive_objects` 確認語法正確、啟用成功，且重新執行 `ZR_RAP09_DEMO` 確認四個既有情境全部沒有回歸問題；但這系統的 `GET PERMISSIONS ONLY INSTANCE FEATURES EML` 語句 ( 官方文件用來 headless 驗證動態欄位控制結果的方式 ) 本身又踩到另一個不支援的語法 ( `GET PERMISSIONS ... ENTITY ... REQUEST ... RESULT ...` 這個完整型式報錯 )，所以這次沒辦法用 `programrun` 直接驗證「Update 時欄位真的變 read-only」這個執行期行為，麻煩你回 **Eclipse Preview** 實際測一次：對一筆既有訂單按 Edit，確認 Order ID ( Header ) 跟 Item ID ( Item，子表格裡的每一列 ) 現在應該是灰階不可編輯的；建立新訂單 / 新增 Item ( `addItem` 按鈕 ) 時這兩個欄位應該還是正常可填。如果畫面跟預期不符，請截圖回報，Claude 再進一步排查。

## Part C：這一課的關鍵發現總結

| 發現                                             | 內容                                                                                                                                                                 |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Determination 在 Capstone 情境的角色                 | 不用每次都寫新的， <code>create</code> 方法本來的自動填值邏輯可以直接擴充，多填一個欄位即可                                                                                                           |
| BDEF 的 <code>validation</code> 語法              | 在 Unmanaged 非 Draft 完全不支援，這一課實際手滑寫了一次確認報錯，印證 rap06 官方文件查證的結論；Validation 一律用手寫邏輯                                                                                    |
| Action 的 <code>RESULT</code> 賦值                | 要用 <code>%key + %param</code> ( 巢狀 )，不能扁平列欄位，扁平寫法會報「找不到這個 component」的誤導性錯誤                                                                                         |
| Action 帶商業規則的寫法                                | 在 Handler Method 裡先用 <code>SELECT COUNT(*)</code> 或類似查詢判斷前置條件，不合法就 <code>APPEND</code> 到 <code>failed / reported</code> 並 <code>CONTINUE</code> ，合法才真的執行狀態變更       |
| Capstone 的物件策略                                 | 延伸既有物件 ( rap08 的訂單 ) 比重新設計資料模型更有效率，也更貼近實務 ( 真實專案的新需求大多是「幫既有 BO 加功能」，不是每次都砍掉重練 )                                                                                    |
| Composition 子實體要不要 <code>expose</code>         | 要——rap08 原本寫「不用另外 <code>expose</code> 」是沒實測過的錯誤推論，被這一課實測推翻：不 <code>expose</code> 子實體， <code>\$metadata</code> 完全不會有 Navigation Property，Object Page 子表格 Facet 不會渲染 |
| 這系統 ( S/4HANA 1909 + Unmanaged 非 Draft ) 的已知限制 | Object Page 子表格的標準 Create 按鈕不可靠 ( Edit 模式下 Delete 消失、Create 不出現、Key 欄位被誤標可編輯 )，已超出這門課「RAP 後端」範圍，不繼續深究 UI5 框架本身                                                     |

| 發現                          | 內容                                                                                                                                                                             |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 解法：把「新增子實體」改成 Custom Action | 用 <code>action ... parameter &lt;Abstract Entity&gt; result [1] \$self;</code> 帶參數的 Action ( <code>addItem</code> ) 繞開不可靠的標準 Create 機制，Fiori Elements 會自動彈出參數輸入視窗，體驗接近、實作路徑更可靠 |

## 學習目標

- 能把 Determination / Validation / Action 三種機制整合進同一個 RAP BO，知道彼此的分工 ( 自動填值 / 資料驗證 / 商業操作 )
- 能寫出「Action 帶商業規則前置條件」的 Handler Method：查詢判斷 + 條件式拒絕 + `failed / reported` 回報
- 記得 Action 的 `RESULT` 要用 `%key + %param` 巢狀賦值
- 知道 BDEF 的 `validation` 宣告式語法在 Unmanaged 非 Draft 不支援，遇到需求要用手寫邏輯達成同樣效果
- 能在 Eclipse ADT 對已有的 RAP 物件 ( Table / CDS View / BDEF / 實作類別 ) 做增量修改，而不是每次都砍掉重練
- 完整走過一次 Table→CDS→BDEF→Service Definition→Service Binding 的全流程，並用 EML / Postman / Fiori Elements 三種方式驗證同一個 BO
- 知道 Composition 子實體想在 Object Page 子表格顯示，Service Definition 也要 `expose` 它，不能只曝露 Root
- 能寫出帶輸入參數的 Action：`action ... parameter <CDS Abstract Entity> result [1] $self;`，知道 Handler Method 用 `%param` 讀取參數、EML 呼叫端用 `%param-<欄位> = ...` 帶入參數
- 知道「標準 Fiori Elements UI 元件不可靠時，改用 Custom Action 繞過」這個實務解法，以及它的取捨 ( 體驗接近但按鈕位置不同 )

## 物件清單

| 物件                         | 名稱                             | 型別                             | 建立方式                                                  | 可執行性                              |
|----------------------------|--------------------------------|--------------------------------|-------------------------------------------------------|-----------------------------------|
| Domain / Data Element      | <code>ZRAP09_ORDSTATUS</code>  | <code>DOMA/DD + DTEL/DE</code> | Claude 建立並啟用 ( 課程獨有業務概念 )                             | —                                 |
| Header 表格 ( 延伸 )           | <code>ZRAP08_ORDER</code>      | <code>TABL/DT</code>           | 使用者 Eclipse 延伸 ( 加 <code>status</code> 欄位 )           | ✓ 已驗證                             |
| Header CDS View ( 延伸 )     | <code>ZI_RAP08_ORDER</code>    | <code>DDLS/DF</code>           | 使用者 Eclipse 延伸                                        | ✓ 已驗證                             |
| Behavior Definition ( 延伸 ) | <code>ZI_RAP08_ORDER</code>    | <code>BDEF/BDO</code>          | 使用者 Eclipse 延伸 ( 加 <code>confirmOrder</code> Action ) | ✓ 已驗證                             |
| 實作類別 ( 延伸 )                | <code>ZBP_I_RAP08_ORDER</code> | <code>CLAS/OC</code>           | 使用者 Eclipse 延伸                                        | ✓ 已驗證                             |
| EML 驗證程式                   | <code>ZR_RAP09_DEMO</code>     | <code>PROG/P</code>            | Claude 建立 ( PROG 不受本課分工限制 )                           | ✓ 已用 <code>programrun</code> 驗證成功 |

| 物件                             | 名稱                | 型別       | 建立方式                                                   | 可執行性  |
|--------------------------------|-------------------|----------|--------------------------------------------------------|-------|
| Metadata Extension (更新)        | ZI_RAP08_ORDER    | DDLX/EX  | Claude 更新並啟用 ( status 標記 + confirmOrder / addItem 按鈕 ) | ✅ 已啟用 |
| Abstract Entity ( addItem 參數 ) | ZI_RAP08_ADDITEM  | DDLX/DF  | Claude 建立並啟用 ( Part D 除錯性質 · 非正課分工 )                   | ✅ 已驗證 |
| Behavior Definition (再延伸)      | ZI_RAP08_ORDER    | BDEF/BDO | Claude 直接延伸並啟用 ( 加 addItem Action )                    | ✅ 已驗證 |
| 實作類別 (再延伸)                     | ZBP_I_RAP08_ORDER | CLAS/OC  | Claude 直接延伸並啟用 ( 加 addItem Handler )                   | ✅ 已驗證 |
| Service Definition (更新)        | ZRAP08_SD         | SRVD/SRV | Claude 更新並啟用 ( 補 expose ZI_RAP08_ORDER_I as Items; )   | ✅ 已啟用 |

**Claude** 用來驗證語法的暫時性物件 ( 不算課程正式物件 · 語法已確認正確 ) : 延伸自 rap08 留下的 ZRAP08V\_H / ZI\_RAP08VU\_H / ZI\_RAP08VU\_I / ZBP\_I\_RAP08VU\_H · 新增 ZR\_RAP09V\_DEMO ( 驗證程式 · 已確認三個情境 + addItem 情境全部成功 ) + ZI\_RAP08VU\_H 的 Metadata Extension ( 已確認 FOR\_ACTION 按鈕語法一次啟用成功 ) + ZI\_RAP08V\_ADDITEM ( Abstract Entity 語法驗證 ) 。

## 驗證方式

1. 你依照上方 Eclipse Step by Step · 延伸 ZRAP08\_ORDER / ZI\_RAP08\_ORDER ( CDS View ) / ZI\_RAP08\_ORDER ( BDEF ) / ZBP\_I\_RAP08\_ORDER
2. 通知 Claude · Claude 讀取比對內容 · 確認 sap\_inactive\_objects 無殘留
3. Claude 建立 ZR\_RAP09V\_DEMO 並用 programrun 執行 · 驗證三個情境 ( 空訂單不能確認 / 正常流程成功 / 不合法 Item 被拒絕 ) 行為正確
4. Claude 更新 ZI\_RAP08\_ORDER 的 Metadata Extension ( status 標記 + confirmOrder 按鈕 ) · 你可以回 Eclipse Preview 確認 status 顯示 · 按鈕出現且點擊行為正確

✅ 已驗收 ( 2026-08-17 ) : 使用者依照 Eclipse Step by Step 完成四個物件的延伸 ( 過程中 Ctrl+1 在既有類別上補新方法沒有跳出快速修正選項 · 改用手動輸入完成 · 符合講義預期的備援方案 ) 。 Claude 讀取比對四個物件 · 內容跟講義完全一致 · sap\_inactive\_objects 確認 0 筆殘留 · ZR\_RAP09V\_DEMO 用 programrun 執行 · 輸出 :

```

=== Scenario 1: confirm order with 0 items -> should FAIL ===
confirm R901 failed: X
  message: Cannot confirm an order with no items
=== Scenario 2: confirm order with 1 valid item -> should SUCCEED ===
create R902+item failed:
confirm R902 failed:
  status after confirm: C
  DB status: C

```

```
=== Scenario 3: create item with quantity <= 0 -> should FAIL at create ===
create R903 header failed:
create R903 item failed: X
    item message: Quantity must be greater than zero
    item rows for R903 in DB:                0
```

三個情境全部符合預期。Claude 接著更新 **ZI\_RAP08\_ORDER** 的 Metadata Extension ( **status** 標記 + **confirmOrder** 按鈕 ) 並啟用成功，**sap\_inactive\_objects** 再次確認 0 筆殘留。

**Part D 加碼 ( Custom Action **addItem** ) 也已驗收**：使用者延伸 **ZI\_RAP08\_ADDITEM** ( Abstract Entity ) 、BDEF ( 加 **addItem** Action ) 、**ZBP\_I\_RAP08\_ORDER** ( 加 **addItem** Handler ) ；Claude 更新 **ZRAP08\_SD** ( 補 **expose Items** ) 與 Metadata Extension ( 加 **addItem** 按鈕 ) ，**sap\_inactive\_objects** 全程確認 0 筆殘留。**ZR\_RAP09\_DEMO** 補上 Scenario 4，**programrun** 執行結果：

```
=== Scenario 4: addItem on an existing order, then confirm ===
create R904 failed:
addItem (bad quantity) failed: X
    message: Quantity must be greater than zero
    item rows for R904 after bad addItem:                0
addItem (valid) failed:
    item rows for R904 after good addItem:                1
    item material_desc:
Good Widget
    quantity:                3.00
confirm R904 (after addItem) failed:
    final DB status: C
```

**rap09** 驗收完成，**rap01 ~ rap09** 全課程正式結案。

## 思考題

- 這一課的 **confirmOrder** 只檢查「有沒有 Item」，如果要再加一條規則「所有 Item 的 **quantity** 總和不能超過 1000」，這條規則該寫在哪裡——**create\_item** ( Item 建立時 ) 還是 **confirmOrder** ( 確認訂單時 ) ？兩種寫法分別會擋住什麼情境，又各自漏掉什麼情境？
- 如果之後要把 **confirmOrder** 之後的訂單設計成「不能再修改 Item」( 例如 Confirmed 狀態的訂單，**update** / **delete** Item 都要被拒絕 )，這個限制該加在哪個方法裡？( 提示：**lhc\_item** 的哪個方法 )
- 這一課的 Determination-equivalent ( **status = 'N'** ) 寫在 **create** 方法裡，如果之後要新增一個「Cancel Order」的 Action ( 把 **status** 改成 'X' )，這個 Action 的 Handler Method 該長什麼樣子？跟 **confirmOrder** 比較，商業規則 ( 能不能被取消 ) 該怎麼設計？

## 答案

這一課的主要物件由使用者延伸 rap08 既有物件，Claude 事後驗證，已於 2026-08-17 驗收完成 ( 含 Part D 的 **addItem** Custom Action 加碼 )。答案物件快照已更新至 **src/ABAP\_Training\_RAP/**：  
**zrap08\_order.tabl.abap** / **zi\_rap08\_order.ddls.abap** / **zi\_rap08\_order.bdef.abap** /  
**zbp\_i\_rap08\_order.clas.locals\_imp.abap** / **zi\_rap08\_order.ddlx.abap** ( 以上五個檔案都已更新為含 rap09 + Part D 內容的最終版本 ) / **zr\_rap09\_demo.prog.abap** ( EML 驗證程式，含 Scenario 4 ) /

zi\_rap08\_additem.ddls.abap ( Abstract Entity ) / zrap08\_sd.srvd.abap ( 已更新為 expose 兩個實體的最終版 ) 。 ZRAP09\_ORDSTATUS Domain / Data Element 已存為 .doma.xml / .dtel.xml 。